

Engineering Standards

GAURI ENGINEERING AND DEVELOPMENT STANDARDS

API DEVELOPMENT STANDARDS

DESIGN PRINCIPLES

Resource-First RESTful Design: APIs are designed as resource-first (nouns, not verbs) using REST conventions unless client's existing architecture requires otherwise (e.g., integrating with existing GraphQL layer)

Semantic Versioning: Breaking changes to published APIs require new version (e.g., /v2/...), never silent changes to existing endpoints

Backwards Compatibility: Non-breaking changes do not require versioning; only addition of new endpoints or major redesigns trigger versioning

Client-Focused API Contracts: API definitions driven by client needs, not internal implementation structure

REQUEST AND RESPONSE STANDARDS

JSON Format: All request and response bodies use JSON

Field Naming: camelCase field names (e.g., customerName, orderTotal, isActive)

Consistent Error Envelope: Every response includes consistent error format for predictable client error handling

Error Structure:

- error.code: Machine-readable error identifier (e.g., INVALID_CUSTOMER_ID, INSUFFICIENT_INVENTORY)
- error.message: Human-readable error description
- error.details: Additional context or validation errors (e.g., specific field validation failures)

Standardised Status Codes: HTTP status codes used consistently across APIs (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error)

AUTHENTICATION AND AUTHORIZATION

OAuth 2.0 Standard: APIs use OAuth 2.0 for authentication when user context exists

Client SSO Integration: Support for client's existing SSO standard (SAML, OpenID Connect)

API Keys for Machine-to-Machine: Only used for integrations with no user context (system-to-system)

Scope and Permission Minimisation: API keys scoped to minimum permissions required for integration (principle of least privilege)

Explicit Role Definition: Every endpoint explicitly defines which roles can access it

No Default Allow: There is no "default allow" for unauthenticated access; all endpoints require explicit authentication

PERFORMANCE AND RELIABILITY

Pagination: Endpoints returning collections must support pagination (offset/limit or cursor) by default; no unbounded result sets

Pagination Standards: Default page size, maximum page size, and cursor formats defined in API documentation

External API Calls: Calls to SAP or Salesforce APIs must implement retry with exponential backoff and circuit breaker pattern

Circuit Breaker: Prevents cascading failures during upstream outages by failing fast when external APIs are unhealthy

Rate Limiting: Client-specified rate limits to prevent abuse and protect infrastructure

Timeout Configuration: All external calls have explicit timeout configuration

Caching Strategy: Caching of SAP and Salesforce data documented and configurable by client

DOCUMENTATION AND MAINTENANCE

OpenAPI (Swagger) Specification: Every API must have OpenAPI/Swagger specification

CI Validation: OpenAPI specification kept in sync with implementation via automated CI validation, not manual documentation

Documentation as Code: Specifications live in version control alongside implementation code

Changelog Maintenance: Changes to APIs documented with date, version, and migration guidance for breaking changes

SDK Generation: OpenAPI specs enable auto-generation of client SDKs in multiple languages (JavaScript, Python, Java, etc.)

CODE REVIEW CHECKLIST AND STANDARDS

BEFORE REQUESTING REVIEW

PR Description: Explain what changed and why (not just "updated code")

Ticket Linkage: Link the relevant tracking ticket/issue

Scope Communication: Clearly state the change scope

User-Facing Evidence: Include screenshots or sample output for user-facing changes

Local Validation: Run full local test suite and formatter before pushing

No Formatting Issues: Review time is for design and logic, not formatting nitpicks (use automated formatter)

WHAT REVIEWERS CHECK

Correctness: Does the change do what it claims, including edge cases and error conditions?

Readability: Could another engineer maintain this code in six months without asking the author?

Scope: Is the PR focused on one change, or does it bundle unrelated work?

Test Coverage: Do tests exercise new behaviour, not just pad coverage numbers?

Security: Any secrets, credentials, or security vulnerabilities introduced?

Performance: Does the change introduce performance regressions?

Dependencies: Are new dependencies necessary and vetted for security?

SECURITY AND DATA HANDLING

Credential Review: Check for secrets or credentials accidentally committed

Input Validation: Ensure all new endpoints validate input against expected types and ranges

Injection Prevention: Prevent SQL injection, command injection, and other input-based attacks

Access Control: Ensure least-privilege access scoping for any new API call or database query

Data Sensitivity: Identify and protect sensitive data (PII, payment info, credentials)

Audit Logging: Ensure security-relevant actions are logged for later audit

GIVING FEEDBACK

Code-Focused Comments: Comment on the code, not the person

Distinction of Severity: Explicitly distinguish blocking issues (must fix before merge) from suggestions (nice to have)

Author Clarity: Authors not left guessing which comments gate the merge

Approval Timing: Approve as soon as blocking issues are resolved

Design Decisions: Review is not the place to relitigate design decisions already agreed in planning

Constructive Tone: Feedback framed constructively to improve code, not demean author

MERGE CRITERIA

Approval Requirement: At least one approval from authorised reviewer

CI Pipeline: Green CI pipeline (all automated checks pass)

Blocking Comments: All blocking comments resolved (suggestions can remain if approved by author)

Self-Merge Authority: Author merges their own PR once criteria are met, unless reviewer explicitly asks to re-review after changes

CODING STANDARDS

GENERAL PRINCIPLES

Readability First: Code should be written to be read by the next engineer, not just to work today

Clarity Over Cleverness: Prefer explicit, clear code over clever, compact code

Single Responsibility: Every function or module should do one thing; if you need "and" to describe what it does, split it

Self-Documenting: Code structure and naming should be self-explanatory

Future Maintainability: Write as if someone else will maintain this code tomorrow

STYLE AND FORMATTING

Automated Formatting: All languages use automated formatter enforcing style (Prettier for JavaScript/TypeScript, Black for Python)

Pre-Commit Hooks: Formatter runs automatically before commit

CI Enforcement: Linting must pass with zero warnings before PR can be opened

Zero Warnings: No warnings tolerated in production code

Style Disputes Settled: Formatter makes style decisions, not debate in code review

Consistency Across Languages: Style consistent within each language ecosystem

NAMING CONVENTIONS

Intent-Based Names: Names describe intent, not implementation (activeCustomers, not list2)

Avoid Abbreviations: No abbreviations except well-known industry terms (ID, URL, HTTP, JSON, SQL)

Class Naming: PascalCase for classes and types (Customer, OrderProcessor, SalesService)

Function Naming: camelCase for functions and methods (calculateTotalPrice, getActiveCustomers)

Constant Naming: UPPER_SNAKE_CASE for constants (MAX_RETRY_ATTEMPTS, DEFAULT_TIMEOUT_MS)

Boolean Naming: Prefix with is, has, or should for clarity (isActive, hasPermission, shouldRetry)

FILE AND FOLDER STRUCTURE

Domain-Driven Structure: Folder structure mirrors domain the code represents, not technical layer

Example: Prefer customers/, orders/, sales/ over utils/, helpers/, common/

Feature Cohesion: Related code grouped together for easy discovery

Avoiding Dumping Grounds: No generic utils/ or helpers/ folders for miscellaneous code

Clear Hierarchy: Nested folders follow logical parent-child relationships

ERROR HANDLING AND LOGGING

Explicit Error Handling: Errors must be handled explicitly and never silently swallowed

User-Facing Errors: Clear, actionable error messages for end users

Internal Logging: Internal errors logged with context (request ID, user ID, relevant identifiers) to debug without reproducing locally

Error Context: Stack traces captured with relevant application state

Structured Logging: Logs include structured metadata (timestamp, severity, component, request ID) for analysis

Production Logging: Appropriate logging level (debug info, warning, error) to balance visibility and performance

Exception Specificity: Catch specific exceptions, not blanket Exception type

TESTING REQUIREMENTS

Unit Testing: New logic requires unit tests covering main path and at least one failure path

Test Scope: Tests verify behavior, not implementation (test what it does, not how it does it)

Test Naming: Test names describe what is being tested (testCalculateTotalPriceWith Multiple Items, testRoundRobinAssignmentWithNoAvailableAgents)

External Integration: Tests for integrations with SAP, Salesforce APIs must run against mocked or sandboxed environments, never live client systems

Test Data: Test data should be realistic but not sensitive (no production customer data in tests)

Regression Testing: Automated tests prevent regression when code is modified

Performance Testing: Performance-sensitive code includes performance benchmarks

DOCUMENTATION REQUIREMENTS

Service README: Every service or module needs a short README covering purpose, setup, and how to run tests locally

Inline Comments: Comments explain why, not what (code shows what; comment explains intent)

Code Smell Signal: Code that needs a comment to explain what it does is a signal to simplify the code instead

Complex Logic: Only complex business logic or non-obvious technical approaches warrant inline comments

API Documentation: Public APIs documented with parameter descriptions, return types, and usage examples

CHANGELOG: Service changes documented in CHANGELOG with date and summary

GIT WORKFLOW AND VERSION CONTROL

BRANCHING MODEL

Trunk-Based Development: Short-lived feature branches merging to main

Always Deployable Main: Main branch always deployable to production (high quality gate)

Feature Branch Naming: feature/short-description (e.g., feature/customer-deduplication)

Bugfix Branch Naming: fix/short-description (e.g., fix/handle-timezone-edge-case)

Branching From Main: Feature and bugfix branches branched directly from main

No Long-Lived Branches: Branches live for days, not weeks; long-lived branches indicate feature is too large

COMMIT STANDARDS

Conventional Commits Format: type(scope): summary

Commit Types: feat, fix, chore, docs, refactor, test

Scope: Optional scope for affected component (e.g., customers, orders, reporting)

Summary: Present tense, imperative ("add customer deduplication" not "added customer deduplication")

Commit Messages Explain Why: Commit messages explain why a change was made, not just what changed (the diff shows what)

Atomic Commits: Each commit represents one logical change; avoid bundling unrelated work in one commit

Commit Frequency: Frequent, small commits easier to review and understand than large, monolithic commits

PULL REQUESTS

Merge Method: Merge via pull request only; direct pushes to main are blocked at repository level

PR Size: Keep PRs small enough to review in under 30 minutes wherever possible

PR Scope: One change per PR (if PR title uses "and," consider splitting)

Automated Checks: PR requires at least one approval and passing CI pipeline before merge (enforced at branch protection)

Conflict Resolution: Rebase onto main to resolve merge conflicts, keeping commit history clean

Self-Merge Authorisation: Author merges their own PR once criteria are met (unless reviewer asks to re-review)

RELEASES AND TAGGING

Semantic Versioning: Releases tagged using semantic versioning (vMAJOR.MINOR.PATCH)

Release from Main: Releases always cut from main branch

Tag Naming: Version tags named vX.Y.Z (e.g., v1.2.3)

MAJOR Version: Increment for breaking changes or major feature releases

MINOR Version: Increment for backwards-compatible new features

PATCH Version: Increment for bug fixes and hotfixes

Release Notes: Release notes document changes since previous version

HOTFIXES

Hotfix Branch: Bugfix branches for released versions branched from release tag (hotfix/brief-description)

Priority Handling: Hotfixes treated with priority to fix production issues quickly

Merge Back: Hotfixes merged back into both the release branch and main to prevent regression

Testing: Hotfixes tested before release to ensure fix doesn't introduce new issues

WORKING WITH CLIENT REPOSITORIES

Client Repository Ownership: For engagements delivered inside client's own repository, follow client's existing branching and review conventions

Gauri Standards Minimum: Gauri's Coding Standards and Code Review Checklist apply as minimum bar regardless of client conventions

Conflict Resolution: Where client conventions conflict with Gauri standards, follow client conventions on branching but maintain Gauri quality standards

Documentation: Gauri standards and client-specific deviations documented in project README

DEPLOYMENT PROCESS AND CI/CD

ENVIRONMENT PROGRESSION

Standard Path: Development → Staging → Production

UAT Environment: Client SAP or Salesforce engagements typically add client-specific UAT environment between Staging and Production

Environment Parity: Each environment as close to production as possible to catch issues early

Secrets Management: Different credentials/secrets for each environment; never reuse production secrets in non-production

CI/CD PIPELINE

Trigger: Every merge to main triggers automated pipeline

Stages: Build → Lint → Unit Tests → Security Scan → Automatic Staging Deployment

Build Stage: Compilation, dependency resolution, artifact creation

Lint Stage: Static code analysis checking for style and potential bugs

Unit Test Stage: Automated unit tests with coverage reporting

Security Scan: SAST (Static Application Security Testing) checking for vulnerabilities

Staging Deployment: Automatic deployment to staging environment after all checks pass

Production Gate: Manual approval gate required before production deployment, even when all automated checks pass

PRODUCTION DEPLOYMENT APPROVAL

Manual Gate: Production deployment requires manual approval from delivery lead or tech lead

Decision Criteria: Approval decision based on change risk, deployment window, and stakeholder readiness

Deployment Authority: Specific engineers granted authority to approve and execute production deployments

Change Records: All production deployments documented in change management system

Rollback Plan Review: Reviewer verifies rollback plan is tested and documented before approving

RELEASE WINDOWS

Change Windows: Production deployments scheduled during agreed change windows

Typical Windows: Outside client business hours for ERP systems, low-traffic periods for customer-facing applications

Exception Path: Critical security fixes may be deployed immediately outside change window with retrospective change-record sign-off

Stakeholder Notification: Client notified of planned deployment window in advance

Deployment Coordination: Deployment coordinated with client operations to ensure readiness

ROLLBACK PLANNING

Mandatory Rollback Plan: Every production deployment must have documented, tested rollback path before approval

Automated Rollback: Preferred method: automated rollback to previous release tag

Schema Changes: Database schema changes must have reviewed, tested down-migration script

Rollback Testing: Rollback plan tested in staging environment before production deployment

Downtime Estimation: Rollback duration estimated and communicated to stakeholders

Quick Rollback: Rollback capability must enable quick reversion if production issues detected

POST-DEPLOYMENT VERIFICATION

Smoke Tests: Deploying engineer runs smoke-test checklist after deployment

Monitoring Window: Monitor error rates and key business metrics for at least 30 minutes post-deployment

Success Criteria: Deployment considered complete only after verification period with no critical issues

Issue Escalation: Any anomalies detected immediately escalated per incident escalation matrix

Go-Live Communication: Results communicated to delivery lead and relevant stakeholders

TESTING STRATEGY

UNIT TESTING

Coverage Target: Minimum 80% code coverage for production code

New Code Coverage: New logic requires unit tests covering main path and at least one failure path

Test Framework: Language-specific testing frameworks (Jest for JavaScript, pytest for Python, JUnit for Java)

Isolation: Unit tests isolated; no external system dependencies

Mocking: External dependencies mocked to test logic in isolation

Execution Speed: Unit tests should complete in seconds (faster feedback loop)

INTEGRATION TESTING

Scope: Testing interactions between components and external systems

SAP Integration: Integration tests against SAP sandbox or demo system, never production

Salesforce Integration: Tests against Salesforce sandbox, never production

Data Validation: Testing data flow end-to-end from source to target system

Error Handling: Testing failure scenarios and retry mechanisms

Cleanup: Test data cleaned up after each test run

SYSTEM TESTING

End-to-End Testing: Testing complete user workflows across multiple systems

UAT Preparation: System tests run in staging and UAT environments before client UAT

Regression Testing: Automated regression tests prevent previously fixed issues from re-appearing

Performance Testing: Performance-sensitive workflows tested for acceptable response times

Load Testing: Critical systems tested with realistic load to identify bottlenecks

PRODUCTION MONITORING

Application Health: Monitoring application uptime, response times, and error rates

Business Metrics: Monitoring key business metrics (orders processed, data quality) for anomalies

Alerting: Alerts triggered for degradation in application or business metrics

Incident Response: On-call rotation to respond to production alerts

Post-Incident Review: Root cause analysis for production incidents